

Алгоритм `corridor_sweeping`: симуляция прохода роя по коридору между двумя движущимися якорями

Описание по коду `main.py` и `point_mass.py`

Аннотация

В файле `corridor_sweeping/main.py` реализована двумерная (по плоскости Oxy) модель роя материальных точек, взаимодействующих только с агентами и двумя *якорями*, попадающими в радиус видимости. Управление задаётся релейно (знак ошибки) с нелинейной функцией $\tanh$, зависящей от расстояний до ближайших соседей по четырём направлениям в относительных координатах. Якоря задают границы коридора и движутся по заданным траекториям $p_L(t)$, $p_R(t)$. Ниже приведены математическая постановка, обозначения с пояснениями при первом появлении и логика шагов симуляции.

Содержание

1	Назначение и общая схема	3
2	Кинематика якорей	3
3	Инициализация агентов	4
4	Множество соседей и состояние <code>peer_state</code>	4
5	Ближайшие соседи по четырём полупространствам	4
6	Закон управления	5
6.1	Вспомогательные функции	5
6.2	Сигналы ошибки	6
6.3	Релейное управление	6
7	Динамика агента (интегрирование)	6
8	Метрики для постобработки логов	7
8.1	Геометрия коридора в плоскости Oxy	7
8.2	Расстояние от агента до прямой через якоря (<code>plot_graph_axis_err</code>) . .	7
8.3	«Интервал» вдоль оси (<code>plot_graph_intervals</code>)	7
9	Логирование и визуализация	8
10	Краткое резюме алгоритма (псевдокод)	8

1 Назначение и общая схема

Цель сценария. Смоделировать поведение N агентов вдоль коридора, образованного двумя параллельно движущимися опорными точками (левый и правый *якорь*). Агенты избегают столкновений и стремятся выстроиться, используя только локальную информацию о соседях в круге радиуса видимости.

Объекты модели.

- **Агент** i — материальная точка с положением $p_i(t) \in \mathbb{R}^3$, скоростью $v_i(t) \in \mathbb{R}^3$ и приложенной силой $u_i(t) \in \mathbb{R}^3$. В коде сила фактически задаёт ускорение при массе $m_i = 1$ (см. раздел 7).
- **Якорь** — неуправляемая опорная точка $a_k(t)$, $k \in \{L, R\}$, движущаяся по заданной функции времени; для агентов якорь ведёт себя как особый сосед в зоне видимости.

Параметры сценария (глобальные константы и аргументы). При первом упоминании ниже переменные сопровождаются пояснением.

- N — `n_agents`: число агентов в рое.
- T — число итераций симуляции `n_iters`; дискретное время $n = 0, 1, \dots, T - 1$.
- Δt — `dt`: шаг интегрирования по времени между обновлениями состояния.
- R_{vis} — `R_vis`: радиус видимости; агент «видит» другого агента или якорь, если евклидово расстояние между ними не превышает R_{vis} .
- u_{max} — `u_max`: модуль горизонтальной составляющей управления по каждой оси Ox и Oy (релейный закон даёт $\pm u_{\text{max}}$).
- w — `w`: положительная константа, подставляемая в функцию g для оси Oy при отсутствии соседа в соответствующем направлении (см. (8)); по оси Ox в коде вместо неё используется 0.
- m_i — `mass` класса `PointMass`: масса агента (по умолчанию $m_i = 1$).

Поток выполнения (process). На каждом шаге n :

1. записать в лог состояния агентов и якорей;
2. обновить время и положения якорей: $a_L(t + \Delta t)$, $a_R(t + \Delta t)$;
3. для каждого живого агента i : найти множество соседей, вычислить управление u_i , применить силу и выполнить шаг интегрирования динамики на Δt .

2 Кинематика якорей

Якорь задаётся функцией движения $f_k : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^3$, $k \in \{L, R\}$ (в коде `left_anchor_motion_func`, `right_anchor_motion_func`).

Переменные.

- t — внутреннее время якоря; на шаге симуляции увеличивается: $t \leftarrow t + \Delta t$ (`AnchorWithFunc.step`).
- $a_k(t) = f_k(t)$ — положение k -го якоря (`anchor.pose`).

Типичный сценарий в конце `main.py`: $f_L(t) = (v_x t, y_L, 0)^\top$, $f_R(t) = (v_x t, y_R, 0)^\top$ с одинаковой скоростью по x и фиксированными y_L , y_R , то есть коридор параллелен оси Ox и весь ансамбль переносится вдоль x .

3 Инициализация агентов

При создании агента i его координата z совпадает с z правого якоря в начальный момент. Координата y выбирается равномерно между y -координатами правого и левого якоря; координата x — равномерно на отрезке, выраженном через положения якорей и R_{vis} (см. исходный код `process`).

Переменные.

- $p_i^0 = (x_i^0, y_i^0, z_i^0)^\top$ — начальное положение i -го агента.
- `id` — целочисленный идентификатор агента для логирования и визуализации.

4 Множество соседей и состояние `peer_state`

Для фокусного агента i формируется список *относительных векторов* к видимым объектам:

$$\mathcal{P}_i = \{ r_{ij} = p_j - p_i : \|r_{ij}\| \leq R_{\text{vis}}, j \neq i, j \text{ жив} \} \cup \{ r_{ik} = a_k - p_i : \|r_{ik}\| \leq R_{\text{vis}} \}. \quad (1)$$

Переменные.

- $r = (r_x, r_y, r_z)^\top$ — элемент списка `peers` (в коде `vec`); компоненты относительного смещения соседа к агенту i .
- $s_i \in \{0, 1, 2\}$ — `peer_state` агента i после вызова `get_peers`:
 - $s_i = 0$, если нет видимых агентов и якорей (кроме самого себя);
 - $s_i = 1$, если есть хотя бы один видимый агент;
 - $s_i = 2$, если в зоне видимости есть хотя бы один якорь.

При наличии нескольких соседей состояние дополнительно обновляется как максимум по видимым агентам: $s_i \leftarrow \max(s_i, s_j)$ для всех видимых агентов j (в коде `peer_state = max(peer_state, agent.peer_state)`).

5 Ближайшие соседи по четырём полупространствам

Для каждого $r = (r_x, r_y, r_z) \in \mathcal{P}_i$ вычисляются четыре «расстояния до ближайшего соседа» в смысле *минимальной положительной компоненты* по осям:

Ось x .

$$d_x^+ = \min\{r_x : r \in \mathcal{P}_i, r_x \geq 0\}, \quad (2)$$

$$d_x^- = \min\{-r_x : r \in \mathcal{P}_i, r_x < 0\} = \min\{|r_x| : r_x < 0\}. \quad (3)$$

Если множество пусто, в коде соответствующее значение остаётся бесконечным (`np.inf`), а флаг «сосед найден» — ложным.

Ось y (аналогично).

$$d_y^+ = \min\{r_y : r \in \mathcal{P}_i, r_y \geq 0\}, \quad (4)$$

$$d_y^- = \min\{-r_y : r \in \mathcal{P}_i, r_y < 0\}. \quad (5)$$

Переменные.

- $d_x^+, d_y^+, d_x^-, d_y^-$ — скаляры, возвращаемые как **distances** (при отсутствии соседа — ∞ в реализации).
- **flags** — четыре булевых признака наличия соседа в каждом из четырёх направлений (`has_peer_x_plus` и т. д.).

Важно. Величины $d_i^\pm$ — это не евклидовы расстояния до соседа, а минимальные проекции со знаком (для «+» — самая маленькая положительная компонента среди соседей в соответствующем полупространстве).

6 Закон управления

6.1 Вспомогательные функции

Функция знака.

$$\text{sgn}(\sigma) = \begin{cases} +1, & \sigma \geq 0, \\ -1, & \sigma < 0. \end{cases} \quad (6)$$

(В коде `sign`; нуль относится к положительной ветви.)

Нелинейность ξ .

$$\xi(g) = \tanh(g). \quad (7)$$

Функция g (замена расстояния при отсутствии соседа).

$$g(d, g_0, h) = \begin{cases} d, & h = \text{истина (сосед есть)}, \\ g_0, & h = \text{ложь}. \end{cases} \quad (8)$$

Здесь d — одно из $d_x^\pm, d_y^\pm$; g_0 — подставляемая константа; h — флаг наличия соседа.

В реализации для оси x всегда $g_0 = 0$; для оси y — $g_0 = w$.

6.2 Сигналы ошибки

Пусть $v_i = (v_x, v_y, v_z)^\top$ — текущая скорость агента i (`agent.dpose`). Определяются скаляры:

$$\sigma_x = v_x - \xi(g(d_x^+, 0, h_x^+)) + \xi(g(d_x^-, 0, h_x^-)), \quad (9)$$

$$\sigma_y = v_y - \xi(g(d_y^+, w, h_y^+)) + \xi(g(d_y^-, w, h_y^-)), \quad (10)$$

где $h_x^+, h_x^-, h_y^+, h_y^-$ — соответствующие флаги наличия соседа.

Интерпретация. Величины $\xi(g(\cdot))$ играют роль желаемых «вкладов» в скорость, зависящих от относительных положений ближайших соседей справа/слева и сверху/снизу в системе координат агента. Разность с фактической скоростью образует ошибку σ_x, σ_y .

6.3 Релейное управление

Горизонтальная сила (ускорение при $m = 1$):

$$u_i = (u_x, u_y, 0)^\top, \quad u_x = -u_{\max} \operatorname{sgn}(\sigma_x), \quad u_y = -u_{\max} \operatorname{sgn}(\sigma_y). \quad (11)$$

Переменные.

- σ_x, σ_y — скалярные сигналы ошибки по осям.
- u_x, u_y — компоненты управления; по Oz в этом сценарии управление нулевое, высота задаётся начальными условиями и не стабилизируется законом (11).

Таким образом, каждая ось независимо подчиняется *bang-bang* закону с насыщением по модулю $u_{\max}$.

7 Динамика агента (интегрирование)

После присвоения $F_i = u_i$ (`apply_force`) выполняется шаг (`step`):

Переменные.

- $a_i = F_i/m_i$ — ускорение (`ddpose`).
- Обновление скорости (явная схема Эйлера для a , затем обновление v):

$$v_i(t + \Delta t) = v_i(t) + a_i(t) \Delta t. \quad (12)$$

- Обновление положения (схема с учётом среднего ускорения в квадратичном члене):

$$p_i(t + \Delta t) = p_i(t) + v_i(t) \Delta t + \frac{1}{2} a_i(t) (\Delta t)^2. \quad (13)$$

Внешняя сила `external_force` в вызове по умолчанию нулевая.

8 Метрики для постобработки логов

При анализе записанных траекторий строятся графики расстояния до оси коридора и «интервала» вдоль оси.

8.1 Геометрия коридора в плоскости Oxy

Пусть $a_L = (x_L, y_L, z_L)^\top$, $a_R = (x_R, y_R, z_R)^\top$ — положения якорей на текущем шаге. Вектор вдоль «оси коридора» в плоскости:

$$\mathbf{a} = (x_L - x_R, y_L - y_R)^\top \in \mathbb{R}^2. \quad (14)$$

8.2 Расстояние от агента до прямой через якоря (plot_graph_axis_err)

Положение агента в плоскости: $\mathbf{p} = (x_i, y_i)^\top$. Вектор от правого якоря к агенту:

$$\mathbf{b} = (x_R - x_i, y_R - y_i)^\top. \quad (15)$$

Проекция $\mathbf{b}$ на направление $\mathbf{a}$ (со знаком, вдоль оси):

$$p_{\parallel} = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|}. \quad (16)$$

Расстояние от точки $\mathbf{p}$ до прямой, проходящей через якорь R в направлении $\mathbf{a}$:

$$d_{\perp} = \sqrt{\|\mathbf{b}\|^2 - p_{\parallel}^2}, \quad (17)$$

что соответствует формуле длины перпендикуляра при не вырожденном $\mathbf{a}$.

Переменные.

- $\|\mathbf{a}\|$ — длина вектора между якорями в плоскости.
- $\|\mathbf{b}\|$ — расстояние от правого якоря до агента в плоскости.
- $d_{\perp}$ — искомое расстояние до оси коридора; строится график $d_{\perp}(t)$ для каждого агента.

8.3 «Интервал» вдоль оси (plot_graph_intervals)

В коде используется величина

$$d_{\text{int}} = \frac{y_i}{\cos \varphi}, \quad \cos \varphi = \frac{a_y}{\|\mathbf{a}\|} = \frac{y_L - y_R}{\|\mathbf{a}\|}, \quad (18)$$

при $\cos \varphi \neq 0$. Это связано с параметризацией положения вдоль наклонной оси при определённой ориентации коридора; при движении якорей параллельно Ox ($y_L \approx y_R$) знаменатель близок к нулю, и величина становится численно неустойчивой — это ограничение конкретной формулы визуализации, а не закона управления.

9 Логирование и визуализация

Запись состояния агента в лог ($n_params = 12$ полей). В каждой строке для каждого агента сохраняются: (x, y, z) , $(\dot{x}, \dot{y}, \dot{z})$, (u_x, u_y, u_z) , `peer_state`, R_{vis} , признак `is_alive`.

GIF (`create_gif`). Строится анимация: траектории якорей, точки агентов, круги радиуса R_{vis} (заливка зоны восприятия).

Переменные визуализации.

- `frame_step`, `start_frame`, `end_frame`, `n_frames` — прореживание кадров и ограничение длины анимации.
- `s_point`, `s_anchor` — размеры маркеров на графике (в пикселях `matplotlib scatter`).

10 Краткое резюме алгоритма (псевдокод)

```
для n = 0 .. T-1:
    записать логи
    для каждого якоря k: t_k += dt; a_k <- f_k(t_k)
    для каждого живого агента i:
        P_i <- относительные векторы к видимым агентам и якорям
        вычислить (d_x^+, d_y^+, d_x^-, d_y^-) и флаги
        sigma_x, sigma_y по формулам из раздела «Закон управления»
        u_i <- (-u_max sgn(sigma_x), -u_max sgn(sigma_y), 0)
        интегрировать v_i, p_i на dt
```

11 Замечания о модели

- Управление (11) разрывное; в связке с $\tanh$ в (9)–(10) получается нелинейная обратная связь по относительным смещениям соседей.
- Якоря не имеют динамики второго порядка в этой симуляции: их положение задаётся кинематически.
- Высота z не регулируется законом управления в `control_force`; сценарий по сути плоский с «заглушкой» по z .